



Fastify

Rotas, Plugins e Arquitetura MVC

Agenda



01 Fastify e seus Diferenciais

O que é, por que usar, instalação e primeiro servidor



02 Rotas e Plugins

Métodos HTTP, validação, hooks, ciclo de vida



03 Arquitetura MVC

Model, Controller, Routes e boas práticas

O que é o Fastify?

Framework web para Node.js focado em performance e baixo overhead. Uma alternativa moderna ao Express.

Criado em 2016

Por Matteo Collina e Tomas Della Vedova

Inspiração

Combina ideias do Express, Hapi e Koa

Ecossistema

200+ plugins oficiais e da comunidade

Principais características:

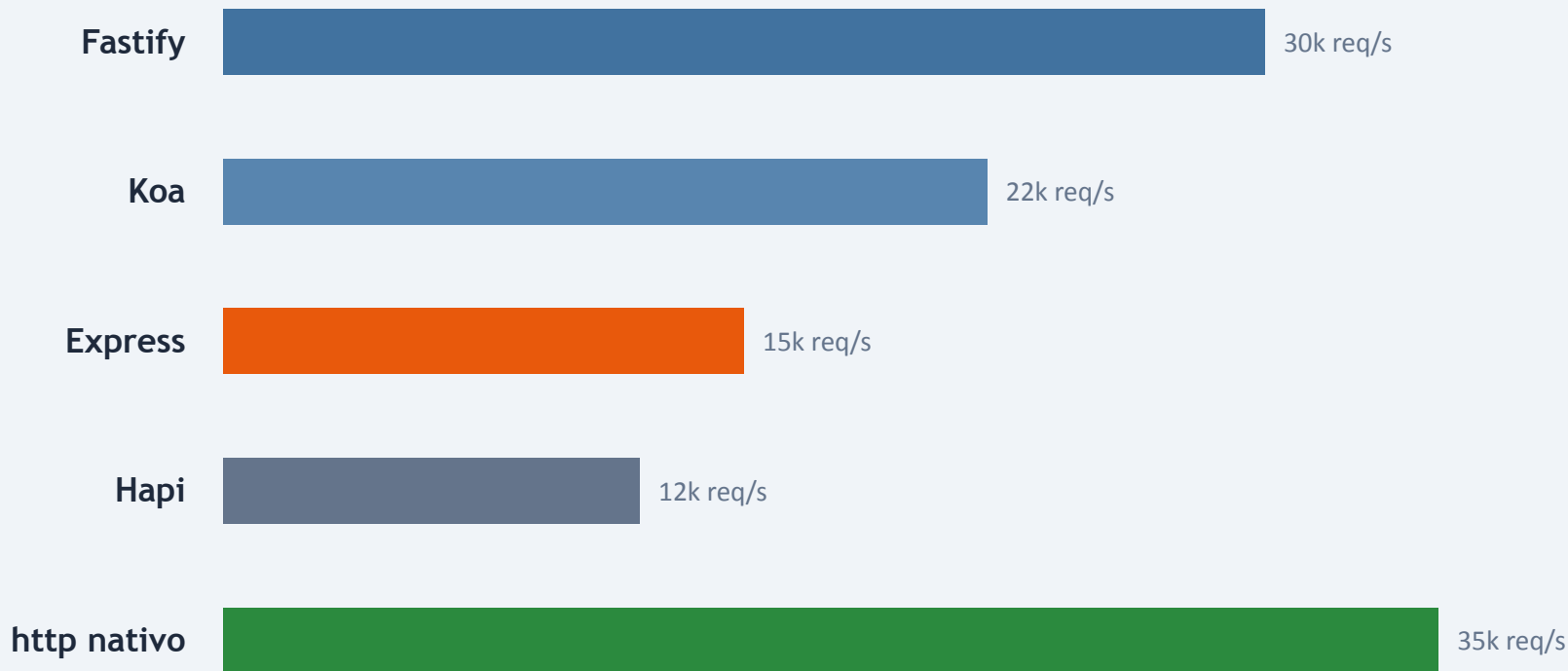
- Até 2x mais rápido que Express em benchmarks
- Validação nativa com JSON Schema (sem bibliotecas extras)
- Sistema de plugins encapsulados (não vazam entre escopos)
- Suporte nativo a async/await
- Logging integrado com Pino (JSON, ultra-rápido)

Por que Fastify e não Express?

Característica	Express	Fastify
Performance	~15.000 req/s	~30.000 req/s
Validação de dados	Manual (Joi, Yup, etc.)	Nativa (JSON Schema)
Parse de JSON	Middleware (body-parser)	Automático
Sistema de extensão	Middlewares (global)	Plugins (encapsulados)
Logging	Externo (Morgan, Winston)	Integrado (Pino)
async/await	Suporte parcial	Suporte nativo
TypeScript	Tipos via @types	Tipos nativos inclusos

Express ainda é o mais popular e tem mais conteúdo online, mas Fastify é a escolha moderna para novos projetos.

Benchmark: Requisições por Segundo



Fonte: benchmarks aproximados (hello world). Valores reais variam conforme a aplicação.

Instalação e Primeiro Servidor

Terminal

```
npm init -y  
npm install fastify
```

```
server.js  
  
const fastify = require('fastify')({  
  logger: true  
});  
  
fastify.get('/', async (request, reply)  
=> {  
  return { mensagem: 'Olá, Fastify!' };  
});  
  
fastify.listen({ port: 3000 }, (err) =>  
{  
  if (err) throw err;  
});
```

Diferenças do http nativo:

Logger automático

Cada requisição é logada automaticamente com Pino

Retorno direto de objetos

Sem `res.end(JSON.stringify(...))`. Basta retornar o objeto.

async/await nativo

Handlers são funções async por padrão

Parse automático de JSON

Body JSON parseado sem middleware extra

Anatomia de uma Rota no Fastify

Estrutura de uma rota

```
fastify.get('/users/:id', async (request, reply) => {  
  const { id } = request.params;  
  const user = buscarUsuario(id);  
  reply.code(200).send(user);  
});
```

request (entrada)

request.params

Parâmetros da URL (:id)

request.query

Query strings (?page=1)

request.body

Corpo da requisição (JSON)

request.headers

Cabeçalhos HTTP

reply (saída)

reply.send(dados)

Envia resposta (JSON automático)

reply.code(201)

Define status code

reply.header('key', 'val')

Define cabeçalho

return { ... }

Atalho: retornar objeto = send()

Rotas e Métodos HTTP

GET, POST, PUT, DELETE e validação

Rotas GET

GET simples

```
// Listar todos
fastify.get('/api/produtos', async () => {
  return { dados: produtos };
});
```

GET com parâmetro (:id)

```
// Buscar por ID
fastify.get('/api/produtos/:id', async (req) => {
  const { id } = req.params;
  const produto = produtos.find(p => p.id === +id);
  if (!produto) throw { statusCode: 404, message: 'Não encontrado' };
  return produto;
});
```

GET com query string

```
// Filtrar e paginar
fastify.get('/api/produtos', async (req) => {
  const { page = 1, limit = 10 } = req.query;
  const inicio = (page - 1) * limit;
  const resultado = produtos.slice(inicio, inicio + +limit);
  return { page: +page, total: produtos.length, dados: resultado };
});
```

Testando no terminal:

```
curl http://localhost:3000/api/produtos
curl http://localhost:3000/api/produtos/1
curl 'http://localhost:3000/api/produtos?page=2&limit=5'
```

Rotas POST

POST - Criar recurso

```
fastify.post('/api/produtos', async (req, reply)
=> {
  const { nome, preco, categoria } = req.body;

  const novoProduto = {
    id: produtos.length + 1,
    nome,
    preco,
    categoria
  };

  produtos.push(novoProduto);
  reply.code(201).send(novoProduto);
});
```

Destakes:

Parse automático do JSON

O Fastify já faz o parse do body automaticamente.
No Express, precisaria de `express.json()` middleware.

Status 201 (Created)

Usar `reply.code(201)` para indicar que o recurso foi criado com sucesso.

Testando com curl:

```
curl -X POST -H "Content-Type:
application/json" -d
'{"nome":"Mouse","preco":99.90}'
http://localhost:3000/api/produtos
```

Rotas PUT e DELETE

PUT - Atualizar recurso

```
fastify.put('/api/produtos/:id', async (req, reply) => {
  const { id } = req.params;
  const index = produtos.findIndex(p => p.id === +id);
  if (index === -1) {
    return reply.code(404).send({ erro: 'Não encontrado' });
  }
  produtos[index] = { ...produtos[index], ...req.body };
  return produtos[index];
});
```

DELETE - Remover recurso

```
fastify.delete('/api/produtos/:id', async (req, reply) => {
  const { id } = req.params;
  const index = produtos.findIndex(p => p.id === +id);
  if (index === -1) {
    return reply.code(404).send({ erro: 'Não encontrado' });
  }
  produtos.splice(index, 1);
  reply.code(204).send();
});
```

Status 204 (No Content): usado quando a operação foi bem-sucedida mas não há conteúdo para retornar.

Validação com JSON Schema

Schema de validação

```
const schemaCriarProduto = {
  body: {
    type: 'object',
    required: ['nome', 'preco'],
    properties: {
      nome: { type: 'string', minLength: 2 },
      preco: { type: 'number', minimum: 0 },
      categoria: { type: 'string' }
    }
  }
};

fastify.post('/api/produtos', {
  schema: schemaCriarProduto
}, async (req, reply) => {
  // req.body já está validado!
  // ...
});
```

Vantagens:

Validação automática

O Fastify valida antes de executar o handler

Erros claros

Retorna 400 com mensagem descritiva automaticamente

Serialização rápida

Schema de response acelera a serialização do JSON

Documentação viva

Schemas podem gerar docs com @fastify/swagger automaticamente

Organizando Rotas em Arquivos

routes/produtos.js

```
async function produtoRoutes(fastify, options) {  
  fastify.get('/api/produtos', async () => { ...  
});  
  fastify.get('/api/produtos/:id', async () => {  
... });  
  fastify.post('/api/produtos', async () => { ...  
});  
  fastify.put('/api/produtos/:id', async () => {  
... });  
  fastify.delete('/api/produtos/:id', async () =>  
{ ... });  
}
```

server.js

```
const fastify = require('fastify')({ logger: true  
});
```

```
fastify.register(require('./routes/produtos'));  
fastify.register(require('./routes/usuarios'));
```

```
fastify.listen({ port: 3000 });
```

Estrutura de pastas:

projeto/

server.js

package.json

routes/

produtos.js

usuarios.js

Cada domínio em seu arquivo. O server.js fica limpo, só registrando plugins.

O que é Middleware?

Middleware é uma função que fica entre a requisição do cliente e a resposta do servidor, podendo interceptar, modificar ou bloquear o fluxo.



Analogia: Segurança de um prédio

Imagine que você quer entrar em um escritório:

- Portaria registra sua entrada (log)
- Catraca verifica seu crachá (autenticação)
- Recepção confirma seu agendamento (validação)
- Só então você chega ao escritório (handler)

Cada etapa pode barrar a entrada! Se o crachá é inválido, você nem chega à recepção.

O que são Hooks?

Hooks são pontos de interceptação no ciclo de vida de uma requisição. Permitem executar código em momentos específicos do processamento.

Middleware (Express)

- Fila linear: um após o outro
- Escopo global por padrão
- Ordem de registro importa muito
- Função com (req, res, next)
- Chamar next() para continuar

Hooks (Fastify)

- Pontos específicos no ciclo de vida
- Escopo encapsulado por plugin
- Cada hook tem um propósito claro
- Função async (request, reply)
- Sem next(); basta retornar ou lançar erro

O que são Decorators?

Decorators são uma forma de adicionar propriedades ou métodos personalizados a objetos existentes (servidor, request ou reply) sem modificar o código original.

Analogia: Acessórios de um carro

O carro (servidor Fastify) vem com funcionalidades de fábrica (rotas, plugins, hooks).

Decorators são como instalar acessórios: você adiciona um GPS (utilitário), sensor de estacionamento (funcionalidade extra) ou porta-copos (conveniência) sem alterar o motor original.

O carro continua funcionando normalmente, mas agora tem capacidades extras.

`fastify.decorate()`

Adiciona ao servidor
Ex: utilitários, config, conexão com banco

`decorateRequest()`

Adiciona ao objeto request
Ex: dados do usuário logado, session

`decorateReply()`

Adiciona ao objeto reply
Ex: métodos de resposta padronizados

Plugins e Hooks

O sistema de extensão do Fastify

O que são Plugins no Fastify?

No Fastify, tudo é plugin. Rotas, decorators, hooks: tudo é registrado via `fastify.register()`.



Encapsulamento

Plugins não vazam para fora do seu escopo. Cada plugin tem seu contexto isolado.



Composição

Plugins podem registrar outros plugins, criando uma árvore de dependências.



Reutilização

Um plugin pode ser usado em múltiplos projetos como um módulo NPM.

Hooks (equivalente a Middlewares)

Hook: log de requisições

```
fastify.addHook('onRequest', async (req) => {
  req.startTime = Date.now();
});

fastify.addHook('onResponse', async (req, reply)
=> {
  const duration = Date.now() - req.startTime;
  console.log(`${req.method} ${req.url} -
${duration}ms`);
});
```

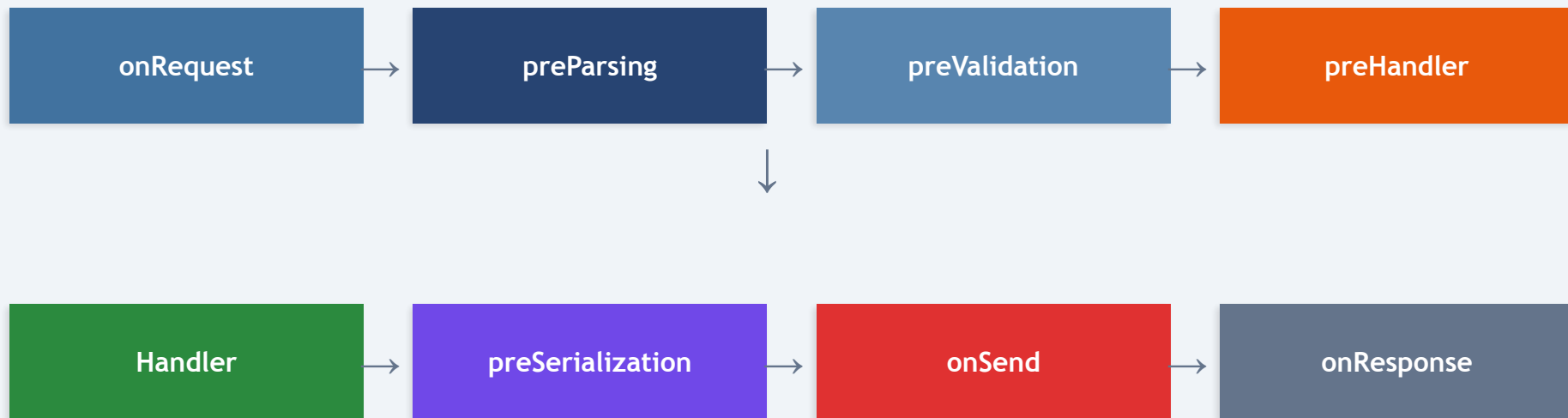
Hook: autenticação simples

```
fastify.addHook('preHandler', async (req, reply)
=> {
  const token = req.headers['authorization'];
  if (!token) {
    reply.code(401).send({ erro: 'Token não
fornecido' });
  }
});
```

Hooks disponíveis:

onRequest	Primeiro a executar
preParsing	Antes do parse do body
preValidation	Antes da validação
preHandler	Antes do handler (auth)
preSerialization	Antes de serializar
onSend	Antes de enviar
onResponse	Após enviar resposta

Ciclo de Vida da Requisição



Cada hook pode modificar o request/reply ou interromper o fluxo (ex: retornar 401 em preHandler).

O Handler é onde sua lógica de negócio roda. Os hooks são para cross-cutting concerns (log, auth, cache).

Plugins Úteis do Ecossistema

@fastify/cors

`npm i @fastify/cors`

Habilita CORS para APIs consumidas por frontends

@fastify/swagger

`npm i @fastify/swagger`

Gera documentação automática da API a partir dos schemas

@fastify/rate-limit

`npm i @fastify/rate-limit`

Limita requisições por IP (proteção contra abuso)

@fastify/env

`npm i @fastify/env`

Registrando plugins

```
fastify.register(require('@fastify/cors'), { origin: '*' });  
fastify.register(require('@fastify/rate-limit'), { max: 100 });
```

Criando seu Próprio Plugin

```
plugins/logger.js

async function loggerPlugin(fastify, options) {
  fastify.addHook('onRequest', async (req) => {
    req.log.info(`${req.method} ${req.url}`);
  });

  fastify.addHook('onResponse', async (req, reply)
=> {
    req.log.info(`Resposta: ${reply.statusCode}`);
  });
}

module.exports = loggerPlugin;
```

```
server.js - usando o plugin

fastify.register(require('./plugins/logger'));
```

Estrutura do plugin:

1. Função async que recebe (fastify, options)
2. Dentro, registrar hooks, rotas, decorators
3. Exportar a função
4. Registrar com fastify.register()

Encapsulamento:

Tudo registrado dentro do plugin fica isolado.
Não afeta outros plugins ou o escopo pai.

Decorators

Adicionando métodos à instância

```
// Decorator na instância do Fastify
fastify.decorate('utilidade', {
  formatarData: (data) =>
    data.toISOString().split('T')[0]
});

// Usar em qualquer handler
fastify.get('/hoje', async function(req) {
  return { data: this.utilidade.formatarData(new
    Date()) };
});
```

Decorator no request

```
// Adicionar propriedade ao request
fastify.decorateRequest('usuario', null);

fastify.addHook('preHandler', async (req) => {
  req.usuario = { id: 1, nome: 'Admin' };
});
```

Tipos de Decorators:

fastify.decorate()

Adiciona ao servidor (utilitários, config)

fastify.decorateRequest()

Adiciona ao objeto request (user, session)

fastify.decorateReply()

Adiciona ao objeto reply

Importante: use function() (não arrow function) para acessar this dentro de handlers com decorators.

Arquitetura MVC

Organizando o projeto em camadas

O que é MVC?



Model

Dados e regras de negócio



View

Resposta JSON da API



Controller

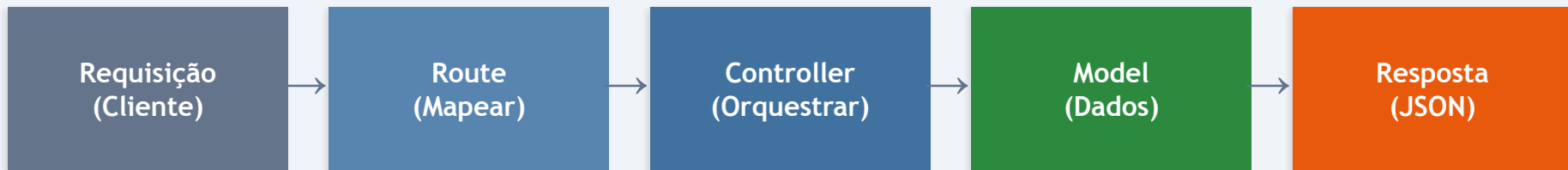
Orquestra request → model → response

Por que usar MVC?

- Separação de responsabilidades: cada camada faz UMA coisa
- Manutenção: alterar o model não quebra as rotas
- Testabilidade: testar cada camada independentemente
- Escalabilidade: adicionar novos domínios sem bagunçar o código

MVC no Contexto de API Back-end

Em APIs, o "View" não é HTML. A View é a resposta JSON formatada que a API retorna.



Route: mapeia URL + método HTTP para o controller correto

Controller: recebe request, chama model, monta response

Model: acessa dados, aplica regras de negócio, retorna resultado

Estrutura de Pastas MVC

```
projeto/  
  src/  
    controllers/  
      produtoController.js  
    models/  
      produtoModel.js  
    routes/  
      produtoRoutes.js  
    plugins/  
      logger.js  
  server.js  
  package.json  
  .env
```

controllers/

Lógica de orquestração (recebe req, chama model, retorna res)

models/

Dados e regras de negócio (CRUD, validações)

routes/

Mapeamento URL → Controller (com schemas)

plugins/

Hooks, decorators, extensões reutilizáveis

server.js

Bootstrap: cria instância, registra plugins e rotas, inicia servidor

Camada Model

```
src/models/produtoModel.js

let produtos = [
  { id: 1, nome: 'Notebook', preco: 3500, categoria: 'Tech' },
  { id: 2, nome: 'Mouse', preco: 120, categoria: 'Tech' },
];

let nextId = 3;

const findAll = () => produtos;
const findById = (id) => produtos.find(p => p.id === id);
const create = (data) => {
  const novo = { id: nextId++, ...data };
  produtos.push(novo);
  return novo;
};

const update = (id, data) => {
  const i = produtos.findIndex(p => p.id === id);
  if (i === -1) return null;
  produtos[i] = { ...produtos[i], ...data };
  return produtos[i];
};
```

```
const remove = (id) => {
  const i = produtos.findIndex(p => p.id === id);
```

Camada Controller

```
src/controllers/produtoController.js
const model = require('../models/produtoModel');

const listar = async (req, reply) => {
  return { dados: model.findAll() };
};

const buscar = async (req, reply) => {
  const produto = model.findById(+req.params.id);
  if (!produto) return reply.code(404).send({ erro: 'Não encontrado' });
  return produto;
};

const criar = async (req, reply) => {
  const novo = model.create(req.body);
  return reply.code(201).send(novo);
};

const atualizar = async (req, reply) => {
  const atualizado = model.update(+req.params.id, req.body);
  if (!atualizado) return reply.code(404).send({ erro: 'Não encontrado' });
  return atualizado;
};
```

```
const deletar = async (req, reply) => {
  if (!model.remove(+req.params.id)) return reply.code(404).send({ erro: 'Não encontrado'

```

Camada Routes

```
src/routes/produtoRoutes.js

const ctrl =
require('../controllers/produtoController');

async function produtoRoutes(fastify) {
  fastify.get('/api/produtos', ctrl.listar);
  fastify.get('/api/produtos/:id', ctrl.buscar);
  fastify.post('/api/produtos', ctrl.criar);
  fastify.put('/api/produtos/:id', ctrl.atualizar);
  fastify.delete('/api/produtos/:id', ctrl.deletar);
}

module.exports = produtoRoutes;
```

Perceba:

- Arquivo limpo e declarativo
- Só mapeia rotas para controllers
- Nenhuma lógica de negócio aqui
- Schemas de validação podem ser adicionados aqui
- Fácil ver todas as rotas de um domínio

```
server.js - registrando
fastify.register(require('../src/routes/produtoRoutes'));
```

Conectando as Camadas



Exemplo: GET /api/produtos/1

1. **Route:** recebe GET /api/produtos/1 → chama `ctrl.buscar`
2. **Controller:** extrai id do params → chama `model.findById(1)`
3. **Model:** busca no array → retorna `{ id: 1, nome: 'Notebook', ... }`
4. **Controller:** recebe o resultado → envia como JSON para o cliente

Refatorando: De 1 Arquivo para MVC

ANTES: tudo em server.js

- Dados, lógica e rotas misturados
- Difícil de manter e testar
- Arquivo cresce indefinidamente
- Impossível trabalhar em equipe

server.js

```
const dados = [...]  
fastify.get(...)  
fastify.post(...)  
// 200+ linhas...
```

DEPOIS: MVC organizado

- Cada arquivo com responsabilidade clara
- Fácil de manter, testar e escalar
- Adicionar domínio = criar 3 arquivos
- Equipe trabalha em paralelo

```
models/produtoModel.js  
controllers/produtoController.js  
routes/produtoRoutes.js  
server.js (10 linhas)
```


Práticas Profissionais

Variáveis de ambiente, tratamento de erros e boas práticas

Variáveis de Ambiente

```
.env
```

```
PORT=3000
```

```
NODE_ENV=development
```

```
DATABASE_URL=mongodb://localhost:27017/app
```

```
server.js - carregando
```

```
require('dotenv').config();
```

```
const PORT = process.env.PORT || 3000;
```

```
const ENV = process.env.NODE_ENV ||  
'development';
```

```
fastify.listen({ port: PORT }, (err) => {  
  if (err) throw err;  
});
```

Por que usar .env?

Nunca hardcodar

Portas, senhas e URLs mudam entre ambientes (dev, prod)

Segurança

O .env nunca vai para o Git (.gitignore)

Instalação:

```
npm install dotenv
```

Tratamento de Erros Centralizado

```
server.js - error handler

fastify.setErrorHandler((error, request, reply) => {
  const statusCode = error.statusCode || 500;

  reply.code(statusCode).send({
    erro: true,
    mensagem: error.message || 'Erro interno do
servidor',
    statusCode
  });
});
```

```
No controller - lançando erros

const buscar = async (req, reply) => {
  const produto = model.findById(+req.params.id);
  if (!produto) {
    const err = new Error('Produto não encontrado');
    err.statusCode = 404;
    throw err;
  }
}
```

Vantagens:

- Um único lugar para tratar todos os erros
- Respostas de erro padronizadas
- Controllers ficam limpos (throw e pronto)
- Fácil adicionar logging de erros

Formato padronizado:

```
{
  erro: true,
  mensagem: '...',
  statusCode: 404
}
```

server.js Final (Bootstrap)

```
server.js - completo e enxuto
require('dotenv').config();
const fastify = require('fastify')({ logger: true });

// Plugins
fastify.register(require('@fastify/cors'), { origin: '*' });
fastify.register(require('./src/plugins/logger'));

// Rotas
fastify.register(require('./src/routes/produtoRoutes'));
// fastify.register(require('./src/routes/usuarioRoutes'));

// Error handler
fastify.setErrorHandler((error, req, reply) => {
  reply.code(error.statusCode || 500).send({
    erro: true,
    mensagem: error.message
  });
});

// Iniciar
const PORT = process.env.PORT || 3000;
```

Boas Práticas

Cada arquivo faz UMA coisa

Model só dados, Controller só orquestração, Route só mapeamento

Nomes descritivos

produtoController.js, não ctrl.js ou pc.js

Sempre async/await

Nunca usar callbacks em handlers do Fastify

Validar entrada com JSON Schema

Nunca confiar nos dados que chegam do cliente

Status codes corretos

200 OK, 201 Created, 204 No Content, 400 Bad Request, 404 Not Found

Tratar erros sempre

setErrorHandler centralizado e try/catch quando necessário

Resumo

01

Fastify

Framework rápido com validação nativa, async/await e logging integrado

02

Rotas e Validação

GET, POST, PUT, DELETE com JSON Schema para validação declarativa

03

Plugins e Hooks

Sistema encapsulado de extensão com ciclo de vida completo

04

Arquitetura MVC

Model (dados), Controller (orquestração), Routes (mapeamento)

05

Práticas Profissionais

Variáveis de ambiente, error handler, organização de projeto